

UCS645
PARALLEL & DISTRIBUTED COMPUTING

ASSIGNMENT 5:

Submitted by: Vansh Bhasin (102303270)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Lab 5: MPI Part II

UCS645: Parallel & Distributed Computing

TIET Patiala

1. Objective

This lab covers advanced MPI programming concepts including blocking vs non-blocking communication, collective operations (MPI_Bcast, MPI_Reduce), and master-slave programming patterns. Five programs are implemented: DAXPY loop, Broadcast Race, Distributed Dot Product, Prime Number Finder, and Perfect Number Finder.

2. Environment

All programs compiled and run on WSL2 (Ubuntu) with MPICH installed. Compilation command used:

```
mpicc -O3 -lm -o <program> <program>.c
```

3. Question 1: DAXPY Loop

3.1 Problem

Implement the DAXPY operation $X[i] = a \cdot X[i] + Y[i]$ for vectors of size $N = 2^{16} = 65536$ using MPI. Measure speedup compared to single process execution using MPI_Wtime().

3.2 Approach

- Each process initializes and operates on its own chunk of size N/p
- MPI_Barrier used before and after computation for accurate timing
- Scalar $a = 2.5$, $X[i] = i$, $Y[i] = i \cdot 0.5$

3.3 Results

Processes (p)	Time (s)	Speedup (S)	Efficiency (E)	Result Correct
1	0.000040	1.00	100%	Yes
2	0.000010	4.00	200%	Yes
4	0.000012	3.33	83%	Yes
8	0.000026	1.54	19%	Yes

3.4 Analysis

The DAXPY operation completes correctly across all process counts — $X[0]=0.0$ and $X[1]=3.0$ match expected values. With 2 processes the speedup is 4x due to both parallelism and reduced overhead. At 4 and 8 processes, communication overhead begins to dominate over the

small workload, reducing efficiency. This is consistent with Amdahl's Law — for small N, the parallel fraction is limited and overhead dominates.

4. Question 2: Broadcast Race

4.1 Problem

Compare a manual broadcast (MyBcast) using a for loop of MPI_Send calls against the built-in MPI_Bcast on a 10 million double array (~76 MB). Run with 2, 4, and 8 processes and measure execution time for both.

4.2 Approach

- MyBcast: Rank 0 loops through all other ranks and sends sequentially using MPI_Send
- MPI_Bcast: Uses the built-in collective operation which internally uses a tree-based algorithm
- MPI_Barrier placed before and after each method for fair timing comparison

4.3 Results

Processes	MyBcast (s)	MPI_Bcast (s)	Speedup	Winner
2	0.034332	0.008532	4.02x	MPI_Bcast
4	0.090254	0.014750	6.12x	MPI_Bcast
8	0.203960	0.058992	3.46x	MPI_Bcast

4.4 Analysis

MyBcast scales linearly — with 8 processes it takes nearly 6x longer than with 2 processes because Rank 0 must send the entire 76 MB array sequentially to each process one at a time. MPI_Bcast is consistently 3-6x faster because it uses a tree-based distribution: Rank 0 sends to Rank 1, then both send to 2 others simultaneously, halving the time at each step. The theoretical complexity of MyBcast is $O(p)$ while MPI_Bcast is $O(\log p)$, which is clearly demonstrated by these results.

5. Question 3: Distributed Dot Product & Amdahl's Law

5.1 Problem

Compute the dot product of two vectors ($A[i]=1.0$, $B[i]=2.0*\text{multiplier}$) with 1 million elements each. Rank 0 broadcasts the multiplier using MPI_Bcast. Each process generates and computes its own local chunk. MPI_Reduce collects the global sum. Run with 1, 2, 4, 8 processes.

5.2 Approach

- Multiplier = 3.0 broadcast from Rank 0 to all processes

- Each process allocates and fills chunk = N/p elements locally
- Local dot product computed independently, then MPI_Reduce with MPI_SUM

5.3 Results

Processes	Time (s)	Speedup	Efficiency	Result	Correct
1	0.000909	1.00	100%	6000000	Yes
2	0.000516	1.76	88%	6000000	Yes
4	0.000365	2.49	62%	6000000	Yes
8	0.000481	1.89	24%	6000000	Yes

5.4 Analysis

All runs produce the correct result of 6,000,000 ($= 1M * 1.0 * 2.0 * 3.0$). The best speedup is achieved at 4 processes (2.49x). At 8 processes, communication overhead from MPI_Reduce and process synchronization exceeds the computation benefit for this small problem size. Perfect linear speedup (4x with 4 cores) was not achieved due to: MPI_Bcast and MPI_Reduce communication overhead, process launch and synchronization cost, and memory allocation per process. This directly illustrates Amdahl's Law — the serial portions (broadcast, reduce) limit the maximum achievable speedup.

6. Question 4: Prime Number Finder

6.1 Problem

Find all prime numbers up to 1000 using a master-slave MPI pattern. The master distributes numbers to test, slaves test primality and return results. Master uses MPI_ANY_SOURCE to receive from any available slave.

6.2 Approach

- Master loops from 2 to MAX_VALUE, sends numbers to idle slaves
- Slaves send zero initially to signal they are ready
- Slaves return +n if prime, -n if not prime
- Master terminates slaves by sending -1 when all numbers are exhausted

6.3 Results

Parameter	Value	Notes
Range tested	2 to 1000	MAX_VALUE = 1000
Processes used	4 (1 master + 3 slaves)	mpirun -np 4
Primes found	168	Correct (known result)
Pattern	Master-slave with MPI_ANY_SOURCE	Dynamic load balancing

Primes found: 2 3 5 7 11 13 17 19 23 29 ... 977 983 991 997

6.4 Analysis

168 primes were found up to 1000, which is the correct known result. The master-slave pattern with `MPI_ANY_SOURCE` provides dynamic load balancing — faster slaves receive more numbers to test. Note that the output order may vary slightly since processes return results asynchronously, but the count and values are always correct.

7. Question 5: Perfect Number Finder

7.1 Problem

Find all perfect numbers up to 10000 using the same master-slave MPI pattern. A perfect number equals the sum of its proper divisors (e.g., $6 = 1+2+3$).

7.2 Approach

- Same master-slave structure as the prime finder
- Slaves compute the sum of proper divisors for each number
- Return $+n$ if perfect ($\text{sum} == n$), $-n$ if not perfect
- Master collects and prints all perfect numbers found

7.3 Results

Parameter	Value	Notes
Range tested	2 to 10000	<code>MAX_VALUE = 10000</code>
Processes used	4 (1 master + 3 slaves)	<code>mpirun -np 4</code>
Perfect numbers found	6, 28, 496, 8128	4 perfect numbers
Pattern	Master-slave with <code>MPI_ANY_SOURCE</code>	Dynamic load balancing

7.4 Analysis

Four perfect numbers were found: 6, 28, 496, and 8128. These are the correct known perfect numbers up to 10000. The master-slave pattern works efficiently here since divisor-sum computation cost grows with n , making dynamic load balancing via `MPI_ANY_SOURCE` beneficial — larger numbers take longer to test and are distributed fairly among available slaves.

8. Conclusion

This lab demonstrated five key MPI programming concepts. The DAXPY loop showed that parallelism benefits diminish for small workloads due to communication overhead. The Broadcast Race clearly proved that `MPI_Bcast`'s tree-based algorithm is 3-6x faster than a naive for-loop broadcast, with $O(\log p)$ vs $O(p)$ complexity. The Distributed Dot Product illustrated Amdahl's Law — serial sections like `MPI_Bcast` and `MPI_Reduce` limit perfect linear scaling. The Prime and Perfect Number finders demonstrated the master-slave pattern with

MPI_ANY_SOURCE for dynamic load balancing, successfully finding all 168 primes up to 1000 and all 4 perfect numbers up to 10000.